

NLP-Based Resume-Job Matching Classification Using TF-IDF and Machine Learning

Carla Radieddine (242747)

In today's competitive job market, efficiently matching candidates to suitable job opportunities has become a critical challenge for organizations. Traditional recruitment processes often rely on manual screening of resumes, which is time-consuming, subjective, and prone to inconsistencies. With the increasing volume of job applications, there is a growing need for automated systems that can assist in identifying suitable candidates based on their qualifications and experience. Natural Language Processing (NLP) provides powerful techniques for analyzing and understanding textual data such as resumes and job descriptions. By leveraging NLP methods, it is possible to extract meaningful patterns from unstructured text and build models that can evaluate the compatibility between a candidate's profile and a job requirement. This project focuses on developing an NLP-based system for resume-job matching classification. The goal is to determine whether a given resume is a good match for a specific job description based on textual information. The dataset includes various resume fields (such as skills, education, and experience) as well as job-related information, along with a matching score that reflects the degree of compatibility. To achieve this, the project implements a complete NLP pipeline including text preprocessing, feature extraction using TF-IDF, and the application of multiple machine learning and deep learning models. The performance of classical models such as Logistic Regression and Random Forest is compared with a deep learning approach based on Long Short-Term Memory (LSTM) networks. The results of this study provide insights into the effectiveness of different modeling approaches for resume-job matching and highlight the strengths of traditional machine learning methods when applied to structured textual data.

The primary objective of this project is to develop a system that can automatically determine whether a candidate's resume matches a given job description using Natural Language Processing techniques. The dataset used in this project contains both resume-related information (such as skills, education, and experience) and job-related information (such as required skills, job responsibilities, and job titles). Additionally, each resume-job pair is associated with a numerical matched score, representing the degree of compatibility between the candidate and the job.

Since the matched score is a continuous value, it was transformed into a binary classification problem to simplify modeling. A threshold of 0.7 was applied to convert the score into two classes:

- Match (1): if the matched score is greater than or equal to 0.7
- No Match (0): if the matched score is less than 0.7

This threshold was used to distinguish strong resume-job compatibility from weaker matches and enabled the use of classification algorithms for prediction.

Formally, the problem can be defined as follows: given a set of textual features describing a resume and a job description, the task is to learn a function that predicts whether the resume is

a good match for the job. The main challenges in this problem include handling unstructured and noisy textual data, extracting meaningful features from long text fields, capturing semantic relationships between resume content and job requirements and managing potential class imbalance in the dataset. To address these challenges, the project applies a structured NLP pipeline involving preprocessing, feature extraction, and model training, followed by performance evaluation using multiple metrics.

The dataset used in this project is publicly available on Kaggle: Resume Dataset for Job Matching (<https://www.kaggle.com/datasets/saugataroyarghya/resume-dataset>)

This dataset contains structured and unstructured textual data representing both candidate resumes and job descriptions. Each record corresponds to a resume-job pair and includes a variety of attributes describing the candidate's profile as well as the job requirements.

The dataset contains 9,544 records, each representing a potential match between a resume and a job. The key target variable is the matched score, which indicates the degree of compatibility between the resume and the job description.

In addition to the target variable, the dataset includes 35 textual features that can be broadly categorized into two groups:

The "Resume-Related" features describe the candidate's background and qualifications, including: career objective, skills, educational institution name, degree names, major field of studies, professional company names, job positions, responsibilities, extra-curricular activities, certifications and languages. These fields contain unstructured text and vary significantly in length and quality.

The "Job-Related" features describe the job requirements and expectations, including: job position name, required skills, job responsibilities, educational requirements, experience requirements. These fields provide the necessary context for evaluating whether a resume is suitable for a given job.

The dataset initially contained missing values in several columns, particularly in optional fields such as extra-curricular activities, certifications, and language proficiency. These missing values were handled during preprocessing by replacing them with placeholder text ("Not Available") to ensure consistency and avoid errors during text processing.

The dataset presents several challenges typical of real-world NLP tasks:

- Unstructured text: Most features consist of free-form text with varying formats.
- Noise and redundancy: Some entries contain repeated or irrelevant tokens such as "name" or "year."
- Imbalanced information: Some resumes are highly detailed, while others are minimal.
- High dimensionality: After feature extraction, the text data results in a large number of features.

These challenges necessitate careful preprocessing and feature engineering to ensure effective model performance.

The dataset can be accessed and downloaded from the Kaggle link provided above. To use the dataset, users need a Kaggle account. After downloading, the CSV file can be loaded into a Python environment using libraries such as pandas.

Exploratory Data Analysis (EDA) was conducted to understand the structure, quality, and characteristics of the dataset before model development. The analysis focused on text length, vocabulary size, token distribution, class balance, and identification of noise and bias.

The distribution of the matched score showed that most resume-job pairs fall within the range of approximately 0.6 to 0.9, indicating moderate to high compatibility. Based on this distribution, a threshold of 0.7 was used to convert the continuous score into a binary classification problem:

- Match (1): score ≥ 0.7
- No Match (0): score < 0.7

After binarization, the dataset showed a relatively balanced class distribution:

- No Match: 5075 samples (53.17%)
- Match: 4469 samples (46.83%)

This mild imbalance does not significantly affect model training and allows the use of standard evaluation metrics.

Text length was analyzed for both resume-related fields and the combined text used for modeling. Resume fields (career objective + skills) contained mostly short to medium-length descriptions. The full combined text had an average length of approximately 104 words, indicating richer contextual information after merging resume and job-related features.

This variation suggests differences in candidate detail levels, which may influence model performance.

The processed corpus contained a vocabulary size of 2385 tokens, indicating moderate lexical diversity. Frequent terms included: "management", "development", "analysis", "data", "science", "learning", "engineering" and "system". These terms reflect the technical and professional nature of the dataset. However, some frequent tokens such as "name" and "year" indicate the presence of noise or placeholder text.

Bigram analysis revealed meaningful domain-specific phrases such as: "machine learning", "data analysis", "computer science", and "technical support". These multi-word expressions provide stronger contextual information than single tokens and highlight common skills, roles, and educational backgrounds. Some redundant patterns (e.g., repeated words) also indicate minor data quality issues.

Visualizations such as word clouds, frequency plots, and histograms showed that: technical skills like "Python, Machine Learning and SQL" are highly frequent, most resumes target data science and engineering roles, text lengths are generally short to moderate and the class distribution is relatively balanced.

The EDA identified several key challenges:

- Noise and placeholders: frequent tokens such as "name" and "year"
- Redundancy: repeated words and phrases
- Domain bias: strong focus on technical and engineering roles
- Variation in text quality: differences in detail across resumes

Overall, the dataset contains meaningful domain-specific patterns relevant to resume-job matching, but also includes noise and variability that require careful preprocessing. These characteristics influenced the choice of feature extraction and modeling techniques used in this project.

Text preprocessing was applied to clean and standardize the dataset before feature extraction and modeling. The goal was to reduce noise, normalize textual content, and improve the quality of input features.

Missing values in all text fields were replaced with a placeholder value ("Not Available") to ensure consistency and avoid errors during text processing and model training.

The text data was processed using standard NLP techniques, including: conversion to lowercase to ensure uniformity, removal of punctuation, special characters and numerical noise, tokenization into individual words, removal of stopwords (such as "the", "and", "is") and lemmatization to reduce words to their base forms. These steps helped reduce variability in the text and improve the consistency of word representations.

Relevant textual fields from both resumes and job descriptions were selected and combined into a single feature called "combined_text". This unified representation allowed the models to learn relationships between candidate qualifications and job requirements.

The resulting text was cleaned and normalized, free of common noise and irrelevant tokens and structured into a consistent format.

This processed text was then used for: TF-IDF vectorization and sequence tokenization and padding.

The preprocessing pipeline ensured that the textual data was standardized and suitable for modeling. It reduced noise, improved consistency, and enabled effective feature extraction for both machine learning and deep learning approaches.

After preprocessing, the textual data was transformed into numerical representations suitable for machine learning and deep learning models. Relevant resume and job-related fields were merged into a single feature called "combined_text". This representation integrates candidate qualifications and job requirements into one input, enabling the model to learn relationships between them.

For classical machine learning models, the TF-IDF (Term Frequency–Inverse Document Frequency) method was used to convert text into numerical features. TF-IDF assigns higher importance to informative and less frequent terms while reducing the weight of common words. This representation is well suited for tasks like resume-job matching, where domain-specific keywords (e.g., skills, job titles, and qualifications) play a significant role.

The TF-IDF vectorizer produced a high-dimensional sparse feature matrix, where each feature corresponds to a term in the vocabulary. A maximum feature limit was applied to control dimensionality and reduce computational complexity.

For the LSTM model, the text was converted into sequences of integer tokens using tokenization. These sequences were then padded to a fixed length to ensure consistent input size across all samples. Unlike TF-IDF, this representation preserves word order, which is important for sequence-based models.

Two types of text representations were used in this project: TF-IDF vectors for Logistic Regression and Random Forest, and tokenized and padded sequences for the LSTM model. This approach allowed each model to use a representation suited to its architecture while enabling a fair comparison between classical and deep learning methods.

To solve the resume- job matching classification task, two classical machine learning models and one deep learning model were implemented: Logistic Regression, Random Forest and

LSTM. This combination enables a comparison between classical machine learning methods and a deep learning approach.

The task was formulated as a binary classification problem, where each resume-job pair was labeled as: Match (1) and No Match (0). The input to all models was the processed textual representation of the resume-job pair, while the output was the predicted class label.

Logistic Regression was used as a baseline classifier because it performs well on high-dimensional sparse text features such as TF-IDF. Random Forest was selected as a non-linear ensemble model capable of capturing more complex relationships between textual features. It also provides feature importance scores, which support interpretability,

To include a deep learning component in the project, a Long Short-Term Memory (LSTM) model was implemented. The LSTM model processes tokenized and padded text sequences and is designed to capture sequential patterns and contextual information in text.

Different models used different text representations: Logistic Regression and Random Forest used TF-IDF features and LSTM used tokenized and padded word sequences.

These three models were selected to provide a meaningful comparison between linear, non-linear, and sequence-based approaches for resume-job matching classification.

The dataset was split into training and testing sets using an 80/20 stratified split, resulting in 7635 training samples and 1909 testing samples. Stratification was applied to preserve the class distribution in both subsets.

The target variable was derived from the original matched score by applying a threshold of 0.7.

For the classical machine learning models, TF-IDF features extracted from the combined text were used as input. Logistic Regression and Random Forest were both trained on this representation using fixed random states for reproducibility.

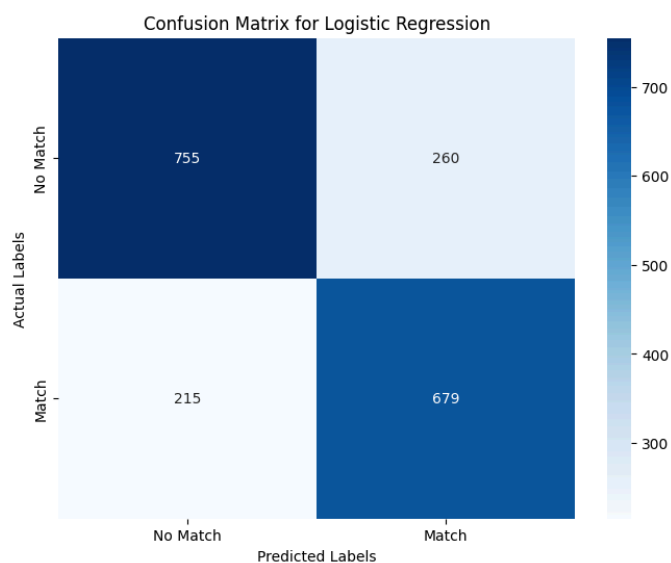
For the LSTM model, the combined text was tokenized, converted into integer sequences, and padded to a fixed length. The model was trained using binary cross-entropy loss, the Adam optimizer, and accuracy as the training metric. Early stopping was applied based on validation loss to reduce overfitting and restore the best-performing model weights.

Hyperparameter optimization was performed on the Random Forest model using RandomizedSearchCV with cross-validation and F1-score as the optimization metric. Although the tuned model achieved a strong cross-validation score, its test-set performance was slightly lower than that of the original Random Forest model. Therefore, the original model was retained as the final best-performing model.

Overall, the training strategy was designed to ensure a fair and reproducible comparison between classical and deep learning approaches.

To evaluate the performance of the models, multiple classification metrics were used, including accuracy, precision, recall, and F1-score. These metrics provide a comprehensive view of model performance, particularly in distinguishing between matched and non-matched resume-job pairs.

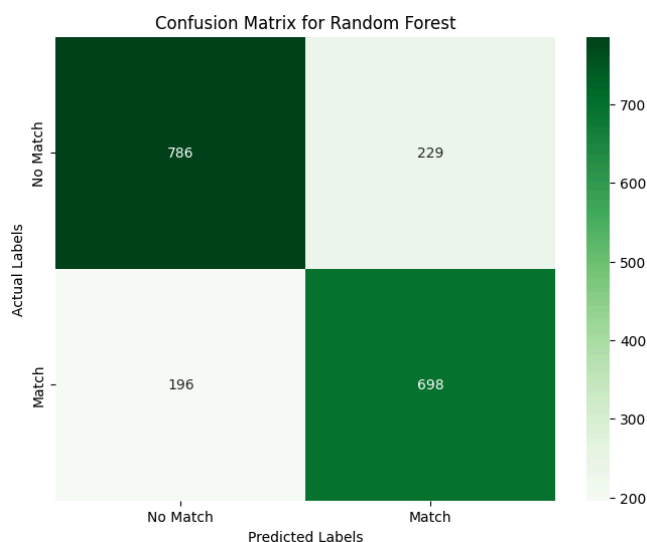
A confusion matrix was also used for each model to analyze prediction behavior in terms of: True Positives (TP), True Negatives (TN), False Positives (FP), False Negatives (FN).



Logistic Regression achieved strong baseline performance on the TF-IDF representation.

- Accuracy: 0.75
- Precision: 0.72
- Recall: 0.76
- F1-score: 0.74

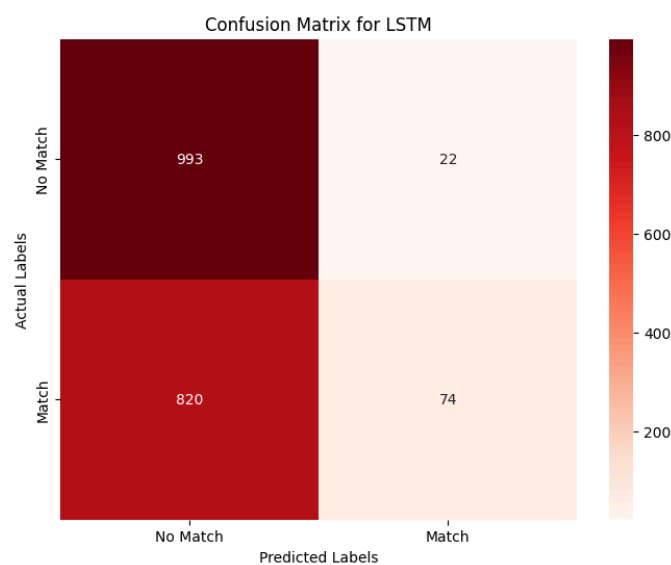
The results show a balanced performance between precision and recall, indicating that the model was able to identify both matches and non-matches effectively. The slightly higher recall suggests that it captured most true matches, with some false positives present. The confusion matrix confirms reasonable performance across both classes.



Random Forest achieved the best overall performance among the evaluated models.

- Accuracy: 0.78
- Precision: 0.75
- Recall: 0.78
- F1-score: 0.77

The model demonstrates a strong balance between precision and recall, with slightly higher recall indicating effective identification of true matches. The confusion matrix shows improved class separation and fewer misclassifications compared to Logistic Regression, confirming its superior performance.



The LSTM model showed significantly weaker performance compared to the classical models.

- Accuracy: 0.56
- Precision: 0.77
- Recall: 0.08
- F1-score: 0.15

The results indicate a major imbalance between precision and recall. While the model achieved high precision, its extremely low recall shows that it predicted very few matches. The confusion matrix confirms that most samples were classified as “No Match,” resulting in many missed true matches.

Overall, Random Forest performed best, followed by Logistic Regression, while LSTM underperformed significantly. These results highlight the effectiveness of classical machine learning models for this task.

To further evaluate model performance beyond numerical metrics, a qualitative error analysis was conducted using the Random Forest model, which achieved the best overall performance. A subset of misclassified samples was examined, including: False Negatives (actual = 1, predicted = 0) and False Positives (actual = 0, predicted = 1).

Several patterns were identified:

- Some false negatives occurred when resumes contained relevant information but lacked strong or explicit keywords, making them harder to detect.
- Some false positives were caused by the presence of common or overlapping terms (such as “management”, “analysis”) which led the model to incorrectly predict a match.
- Generic or noisy tokens such as “available” reduced the model’s ability to distinguish between meaningful and irrelevant content.
- The model relies heavily on keyword overlap and may struggle when similar concepts are expressed using different terminology.

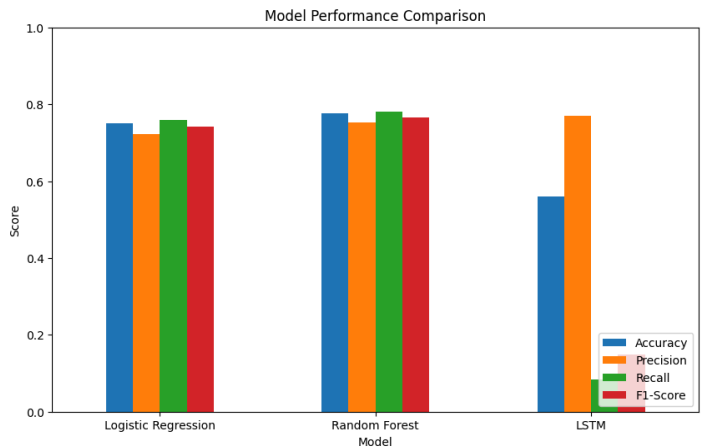
Overall, the error analysis shows that the model performs well when clear domain-specific terms are present, but struggles with ambiguous or less descriptive text. This highlights a limitation of TF-IDF-based approaches, which do not fully capture semantic relationships.

This section compares the performance of the three models: Logistic Regression, Random Forest, and LSTM.

The performance of all models is summarized below:

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.75	0.72	0.76	0.74
Random Forest	0.78	0.75	0.78	0.77
LSTM	0.56	0.77	0.08	0.15

A bar chart is also included to visually compare model performance across these metrics.



Among the three models, Random Forest achieved the best overall performance, with the highest accuracy, recall, and F1-score. It provided the most balanced results across all evaluation metrics. Logistic Regression also performed well and served as a strong baseline. Its performance being close to Random Forest indicates that the task is largely influenced by linear separability of textual features.

In contrast, the LSTM model performed significantly worse. Although it achieved relatively high precision, its extremely low recall indicates that it failed to identify most of the actual matches, making it unsuitable for this task in its current configuration.

The superior performance of the classical models can be explained by the nature of the dataset. Resume-job matching in this dataset relies heavily on explicit keyword overlap, such as skills, job titles, and educational terms. TF-IDF-based representations are well suited for capturing these patterns.

The LSTM model, on the other hand, relies on sequential learning and did not benefit from strong contextual signals in the dataset. Its performance was likely affected by: limited dataset size, presence of noisy and repetitive tokens and lack of retrained embeddings.

Overall, Random Forest was the most effective model, followed closely by Logistic Regression. The results show that classical machine learning approaches outperform the implemented deep learning model for this dataset and task.

Hyperparameter optimization was applied to the Random Forest model using RandomizedSearchCV to explore different parameter combinations efficiently. The following parameters were tuned: `n_estimators`, `max_depth`, `min_samples_split`, and `min_samples_leaf`. The best configuration achieved an F1-score of 0.7688 during cross-validation.

When evaluated on the test set, the tuned model achieved:

- Accuracy: 0.76
- Precision: 0.72
- Recall: 0.80

- F1-score: 0.76

Although the tuned model improved recall, its overall performance was slightly lower than the original Random Forest model. Therefore, the original model was retained. This result highlights that improved cross-validation performance does not always lead to better generalization on unseen data.

Interpretability was addressed using feature importance from the Random Forest model.

The most important features included terms such as: “learning”, “management”, “engineering”, “machine learning”, “development”, and “science”.

These features reflect relevant skills, job roles, and technical domains, indicating that the model relies on meaningful patterns. However, some noisy tokens such as “name” and “year” also appeared, suggesting the presence of residual noise in the dataset.

Key observations include: domain-specific keywords strongly influence predictions, keyword overlap between resumes and job descriptions is critical and noisy tokens can affect model decisions. Limitations of feature importance cover: does not explain individual predictions and does not capture semantic relationships between words. More advanced methods such as SHAP or LIME could provide deeper insights. Overall, the model relies on meaningful textual features but remains sensitive to noise and keyword-based patterns.

The use of NLP in resume-job matching raises important ethical concerns, particularly related to bias, fairness, and responsible use.

The dataset is biased toward technical roles, which may limit the model’s ability to generalize to other domains. Additionally, the model may favor candidates who use specific keywords, potentially disadvantaging those with equivalent skills described differently.

This system should be used as a decision-support tool, not as a fully automated hiring system. Human oversight is necessary to ensure fair and accurate decisions. The model relies heavily on textual patterns and may be influenced by noisy or incomplete data. As a result, predictions should be interpreted cautiously.

While NLP can improve recruitment efficiency, it must be applied responsibly to avoid biased or unfair outcomes.

To ensure reproducibility, fixed random seeds were used for data splitting and model training. A consistent pipeline was applied across all steps, including preprocessing, feature extraction, and model training. The project was implemented using Python with libraries such as pandas, numpy, scikit-learn, NLTK, and TensorFlow/Keras.

To reproduce the results of this project, the following steps should be followed:

1. Download the dataset from Kaggle
2. Load the data into a notebook environment
3. Install required libraries
4. Run the notebook sequentially

Although classical models are fully reproducible, minor variations may occur in deep learning results due to hardware and backend differences.

This project developed an NLP-based system for resume-job matching classification using both classical machine learning and deep learning approaches.

The results showed that Random Forest achieved the best overall performance, followed closely by Logistic Regression. In contrast, the LSTM model underperformed significantly, mainly due to its inability to capture useful patterns from the dataset. These findings indicate that TF-IDF-based classical models are highly effective for this task, as resume-job matching relies strongly on keyword overlap and domain-specific terminology.

The analysis also highlighted important dataset characteristics, including the presence of noise, repeated tokens, and domain bias toward technical roles. Despite these challenges, the models were able to learn meaningful patterns, particularly when clear and relevant keywords were present.

From an ethical perspective, the project emphasized that such systems should be used as decision-support tools rather than fully automated solutions, due to potential bias and limitations in understanding context.

Future improvements could include: using larger and more diverse datasets, applying pretrained embeddings or transformer-based models, improving noise removal and text normalization and incorporating advanced explainability techniques such as SHAP or LIME. Overall, the project demonstrates that well-designed classical NLP approaches can outperform more complex deep learning models when applied to structured, keyword-driven text classification tasks.